

Iteration

CS 101 Algorithmic Problem Solving

Fall 2024

1 What is it?

Some of you might be familiar with the concept of iteration, but we're sure it's brand new to a lot of you. Iteration, in terms of code, is the process of repeating an action over and over. Iteration usually takes place over a range of numbers, until a certain condition is satisfied, or over an *iterable*. A data type which can be iterated over is called an iterable. Some such types are: `string`, `list`, `dictionary`, `tuple`, `set`.

There are two main looping constructs in python: `for` loop and `while` loop.

2 Why do we need it?

The reason you'd want to use iteration is quite simple. Say, you want to repeat a process a set amount of times, like printing "Hello world" 10 times. Without iteration, you'd have to write 10 `print` statements (or perhaps a single, long `print` statement). However, as you will see later, you can easily accomplish this in merely two lines using iteration.

But the uses of iteration don't stop there. Suppose you want to check how many non-alphabetic characters are there in a string. You can easily do that using iteration as well.

In simpler terms, you are to use iteration whenever you need to repeat an action a number of times, or when you need to traverse over something (called *looping*) to check for a certain result.

3 `for` loop

3.1 `range`

`for` loop is typically used when you are iterating over a range of numbers or traversing an iterable. Following is the format of `for` loop for iterating over a range of numbers:

```
for counter in range(start, stop, step): # start, stop, step are integers
    # place code block here
```

This may look a bit confusing so let's take a look at a couple of examples:

```
for i in range(0,4,1):
    print(i)
```

The above code outputs:

```
0
1
2
3
```

Now you may notice, the values of `i` go from `start` to `(stop - 1)`. This is always the behavior of `range`; the `start` argument is *inclusive*, and `stop` argument is *exclusive*. So the values of `i` will always stop right before `stop`.

Another thing to know is that the `start` and `step` arguments are optional. If they are not specified, their default values are 0 and 1 respectively. So we can write the above `for` loop as the following for the same output:

```
for i in range(4): #this is how you write a for loop without start and step
    print(i)
```

Note that if you only specify two arguments, Python will consider the first one as `start` and the second one as `end`.

So the `start` argument is where the values of `counter` begin and it goes till `(stop - 1)`, but what does `step` do?

The `step` argument is basically a way to tell your `for` loop how many “steps” to take when updating `counter`. Observe the following code:

```
for i in range(0,10,2): #must specify all three arguments if using non-default
                        step
    print(i)
```

It outputs the following:

```
0
2
4
6
8
```

As you see, `i` now updates by 2 in each iteration. You can try and imagine how this will look with a `step` of 3, 4, or even 10.

Remember how we talked about printing 10 lines of “Hello world” in 2 lines? The following code does just that.

```
for i in range(10):
    print(i, 'Hello world')
```

As we said, it outputs the following.

```
0 Hello world
1 Hello world
2 Hello world
3 Hello world
4 Hello world
5 Hello world
6 Hello world
7 Hello world
8 Hello world
9 Hello world
```

3.2 Iterable and Indexing

Thus far you’ve seen how you can use a `for` loop over a certain `range`, but what about using it to traverse an iterable, like a `str`?

One way is to directly traverse the characters of the `str` as follows.

```
for counter in iterable:
    # place code block here
```

Let's take a look at an example:

```
st = 'Hello world'
for char in st:
    print(char)
```

The above code outputs the following:

```
H
e
l
l
o

w
o
r
l
d
```

This example was straightforward enough, the counter just moves over each individual character in the string and the code block prints it.

Now for the another, slightly more complicated but very useful method to iterate over an iterable. It involves **indexing**.

Indexing Every item in an iterable has a unique numeric *address* which can be used to identify or locate it within the iterable. This address is called the *index*. Using indexes in a string, you can access specific characters in the string. The first character has an index of 0, the second character's index is 1, and so on. For instance, in the string "Hello", the character "H" is at index 0, "e" is at 1, and so on.

In python, the syntax for string indexing involves using square brackets after the string like in the following example.

```
st = 'Hello'
print(st[0], st[1], st[4])
```

The above code outputs the following:

```
H e o
```

3.3 Using `for` loop with indexing

The way you use indexing with `for` loop is by iterating over the range of numbers that correspond to the indexes of the string that you want to iterate over.

Observe the following example for how you can iterate over the entire length of the string:

```
st = 'Hello world'
for i in range(len(string)): # values of i stop at len(string) - 1
    print(i, st[i])
```

The above code outputs:

```
0 H
1 e
2 l
3 l
4 o
5
6 w
7 o
8 r
9 l
10 d
```

Now suppose you only want to iterate from string index 6 to index 9, you'd do it as follows:

```
st = 'Hello world'
for i in range(6, 10): # values of i stop at 9
    print(i, st[i])
```

This will output the following:

```
6 w
7 o
8 r
9 l
```

4 while loop

while loop is quite a fair bit different from **for** loop in terms of syntax and use cases. Let's take a look at the syntax:

```
while condition:
    # place code block here
```

Let's take a look at some examples.

The first example below is a similar use case as the **for** loop; we want to output a string a fixed number of times.

```
count = 0
while count < 10:
    print(count, 'Hello world')
    count += 1
```

The corresponding output is:

```
0 Hello world
1 Hello world
2 Hello world
3 Hello world
4 Hello world
5 Hello world
6 Hello world
7 Hello world
8 Hello world
9 Hello world
```

Though the behavior of the **while** loop above is similar to that of a **for** loop, there are some important differences in syntax. Unlike the **for** loop, you have to assign an initial value to the counter. You also have to increment it yourself.

Let's take a look at another example.

```
st1 = 'He'
st2 = 'Hello world'
ind = len(st1)

while st1 != st2:
    print(st1)
    st1 += st2[ind]
    ind += 1
print(st1)
```

Don't be intimidated by the above block of code. Take your time to understand it if you're having trouble. We have two string variables, `st1` and `st2`, and the `while` loop runs until the two strings become equal. The code outputs the following:

```
He
Hel
Hell
Hello
Hello
Hello w
Hello wo
Hello wor
Hello worl
Hello world
```

There is a fundamental difference in the use of the `while` loop in the two examples. In the first one, its use was similar to a `for` loop - it was used to repeat a block of code a fixed number of times. In the second example, the number of times to repeat is not specified. The block of code keeps on repeating until the condition becomes `False`.

5 `break` statement

If you want to interrupt the iteration process before it completes by itself, you can use a `break` statement which immediately terminates the current loop. Take a look at the following examples:

```
for i in range(10):
    print(i)
    break
```

The above code outputs one line and then the loop breaks as seen below:

```
0
```

Usually you would use a condition for `break` so that your loop doesn't just `break` on the first iteration, like as follows:

```
string = 'Hello world'
char = ' '
for ch in string:
    if ch == char:
        print(char, 'found in string')
        break
print(ch)
```

The above loop breaks as soon as it finds an instance of `char` in `string`.

```
H
e
```

```
1
1
o
    found in string
```

6 Using `for` loop vs. `while` loop

So by now you might be wondering when to use these two different looping constructs. Generally, a `for` loop is used when you know the number of iterations and a `while` loop is used when you don't know the exact number of iterations required but have a certain condition for when the iteration should end.

Nested Loops

Much like how you can nest conditionals, you can also nest loops inside other loops. Here is an example:

```
for i in range(3):
    for j in range(3):
        print(i, j)
    print() # prints an empty line
```

The above code outputs the following:

```
0 0
0 1
0 2

1 0
1 1
1 2

2 0
2 1
2 2
```

You can use any combination of loops that might be required to accomplish your task. However, in case of multiple counters, it is important to keep track of which counter corresponds to which loop.